BRD – Banking Loan Management System (BLMS)
Database-Only Testing Scope

---

1. Document Purpose

This document defines the **database requirements** for the Banking Loan Management System (BLMS).
It is intended for **QA testers** to validate:

- Data integrity
- Business rules
- Relational consistency
- Triggers and constraints

Testing is **limited to INSERT and UPDATE operations only**.

---

2. Project Scope

In-Scope:

- Customer accounts
- Loans and repayments
- Collaterals
- Branches and employees
- Transactions, fees, and penalties

Out-of-Scope:

- Front-end UI
- APIs
- Reporting dashboards

Testing Scope:

- INSERT and UPDATE queries only
- SQL validation of business rules
- Data integrity checks

---

3. Functional Requirements

FR-1 – Customer Management

- Add new customers
- Update customer info
- Mandatory fields: full_name, date_of_birth, email (unique), phone
- Constraints: email must be unique, phone must be numeric

FR-2 – Account Management

- Customers can have multiple accounts
- Account types: Savings, Current, Loan Account

- Mandatory fields: account_number (unique), account_type, balance >= 0
- Business rules:
  - Savings/Current accounts cannot have negative balance
  - Loan accounts balance = total disbursed loan amount

FR-3 – Loan Management

- Add new loans linked to a customer and account
- Mandatory fields: loan_id (PK), account_number, loan_type, principal_amount, interest_rate, start_date, term_months
- Constraints:
  - principal_amount > 0
  - interest_rate > 0
- Business rules:
  - Cannot approve loan if customer has overdue loans
  - Loan balance = principal + interest
  - Repayment schedule auto-generated via trigger

FR-4 – Collateral Management

- Collateral linked to loan
- Mandatory fields: collateral_id, loan_id, type, value
- Business rules:
  - Collateral value ≥ 50% of principal
  - Collateral type ENUM: Property, Vehicle, Gold, Other

FR-5 – Transaction Management

- Track all deposits, withdrawals, loan disbursements, repayments
- Mandatory fields: transaction_id, account_number, transaction_type (Deposit, Withdrawal, Loan Disbursement, Repayment), amount > 0, transaction_date
- Constraints:
  - Cannot withdraw more than balance for Savings/Current accounts
  - Loan repayments cannot exceed outstanding balance

FR-6 – Fees and Penalties

- Auto-calculated via trigger for late repayment or overdraft
- Mandatory fields: fee_id, account_number/loan_id, fee_type, amount
- Business rules:
  - Penalty interest applies 5% after due date
  - Fee cannot exceed 10% of transaction

FR-7 – Employee and Branch Management

- Employee linked to branch

- Branch mandatory fields: branch_id, name, location
- Employee mandatory fields: employee_id, full_name, branch_id
- Constraints: branch_id FK
- Business rules: employee cannot be deleted if involved in any loan approval

---

4. Non-Functional Requirements

- All PKs auto-incremented
- FK relationships enforced
- CHECK constraints for numeric fields, ENUM types
- Triggers to calculate loan repayment schedules, fees, and penalties
- Performance: INSERT/UPDATE ≤ 2s
- DB: MySQL / SQL Server
- Naming convention: snake_case

---

5. Assumptions

- One bank instance
- Dates in YYYY-MM-DD format
- Interest calculated monthly
- Only INSERT/UPDATE operations performed by testers

---

6. Database Schema (SQL Creation)

6.1 Customers

```
CREATE TABLE Customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    date_of_birth DATE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(20) NOT NULL
);
```

6.2 Accounts

```
CREATE TABLE Accounts (
    account_number INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    account_type ENUM('Savings','Current','Loan') NOT NULL,
    balance DECIMAL(15,2) CHECK(balance >= 0),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
```

6.3 Loans

```
CREATE TABLE Loans (
    loan_id INT PRIMARY KEY AUTO_INCREMENT,
    account_number INT NOT NULL,
```

```
    loan_type ENUM('Personal','Home','Auto','Business') NOT NULL,
    principal_amount DECIMAL(15,2) CHECK(principal_amount > 0),
    interest_rate DECIMAL(5,2) CHECK(interest_rate > 0),
    start_date DATE NOT NULL,
    term_months INT CHECK(term_months > 0),
    status ENUM('Active','Closed','Default') DEFAULT 'Active',
    FOREIGN KEY (account_number) REFERENCES Accounts(account_number)
);
```

6.4 Collaterals

```
CREATE TABLE Collaterals (
    collateral_id INT PRIMARY KEY AUTO_INCREMENT,
    loan_id INT NOT NULL,
    type ENUM('Property','Vehicle','Gold','Other') NOT NULL,
    value DECIMAL(15,2) CHECK(value >= 0),
    FOREIGN KEY (loan_id) REFERENCES Loans(loan_id)
);
```

6.5 Transactions

```
CREATE TABLE Transactions (
    transaction_id INT PRIMARY KEY AUTO_INCREMENT,
    account_number INT NOT NULL,
    transaction_type ENUM('Deposit','Withdrawal','Loan Disbursement','Repayment')
NOT NULL,
    amount DECIMAL(15,2) CHECK(amount > 0),
    transaction_date DATE NOT NULL,
    FOREIGN KEY (account_number) REFERENCES Accounts(account_number)
);
```

6.6 Fees

```
CREATE TABLE Fees (
    fee_id INT PRIMARY KEY AUTO_INCREMENT,
    account_number INT,
    loan_id INT,
    fee_type ENUM('Overdraft','Late Repayment') NOT NULL,
    amount DECIMAL(10,2) CHECK(amount > 0),
    FOREIGN KEY (account_number) REFERENCES Accounts(account_number),
    FOREIGN KEY (loan_id) REFERENCES Loans(loan_id)
);
```

6.7 Branches

```
CREATE TABLE Branches (
    branch_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    location VARCHAR(100)
```

);

6.8 Employees

```
CREATE TABLE Employees (
    employee_id INT PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    branch_id INT NOT NULL,
    FOREIGN KEY (branch_id) REFERENCES Branches(branch_id)
);
```

---

7. Realistic Business Rules for DB Testing

- Loan cannot be disbursed if customer has overdue loan
- Collateral value ≥ 50% of principal
- Transaction cannot overdraw balance (for Savings/Current)
- Loan repayment schedule auto-calculated
- Fees/penalties applied via triggers
- Email uniqueness enforced
- Account and loan balances updated via INSERT/UPDATE only

---

8. Testing Guidelines

- INSERT sample data for Customers, Accounts, Loans, Collaterals, Transactions, Fees, Branches, Employees
- UPDATE queries to test business rules and constraints
- Validate using SELECT queries:
    - Loan balances
    - Collateral values
    - Transaction totals
    - Fee/penalty calculations
    - Overdue loans
- Focus on **edge cases**: negative amounts, duplicate emails, overdraft attempts, collateral < 50%

---

9. Complexity & Sizing

- Tables: 8
- Constraints & business rules: 25+
- Triggers for loan schedule and fees: 2
- Estimated test cases: 50–60 (positive + negative)
- Weighting: multi-table relationships → weight 2, simple field constraint → weight 1